Sistema de Gestión de Gimnasio (MVC)

Reporte Técnico Universitario de Análisis, Diseño Modular y Control Operativo

1. FICHA TÉCNICA DEL PROYECTO

Denominación Oficial	Sistema Web Modular de Gestión de Gimnasio bajo Patrón Arquitectónico MVC
Equipo de Desarrollo	• **Harrison Ospina Salazar** (Coordinador y Líder de Integración) • Juan David Bedoya Carmona | • Caleb Alejandro Mora Mejía | • Maicol Steven Laverde Ocampo
Repositorio de Código	`https://github.com/harrisonospina97/proyecto-01-clase.git`
Pila Tecnológica	JavaScript ES6+ (Vanilla), Node.js Runtime (módulo HTTP core nativo), CSS3 modular y DOM API.
Entorno de Ejecución	Servidor HTTP local en `index.js` alojado en puerto 3000 (`http://localhost:3000`). Arquitectura sin dependencias npm de terceros.

2. CONTEXTO DE DOMINIO Y PLANTEAMIENTO DEL PROBLEMA

La administración convencional en centros deportivos opera habitualmente mediante métodos analógicos tales como planillas impresas, cuadernos contables y listas dispersas de asistencia. Dicha dinámica genera vulnerabilidades operativas críticas:

- **Falta de control en accesos:** Ingreso desapercibido de usuarios con cuotas vencidas o planes inactivos.
- **Descontrol de aforo:** Inexistencia de mecanismos en tiempo real para verificar la capacidad máxima de las instalaciones.
- **Desarticulación técnica deportiva:** Falta de registro estructurado y seguimiento periódico de métricas antropométricas (IMC) y rutinas de ejercicio.
- **Incumplimiento normativo (Ley 1581 de 2012):** Omisión del consentimiento expreso y formal para el tratamiento de datos personales y sensibles de salud de los afiliados.

> ***Solución propuesta:*** *Aplicación web modular basada estrictamente en el patrón arquitectónico Modelo-Vista-Controlador (MVC), programada en JavaScript nativo sobre el entorno Node.js, con almacenamiento reactivo en memoria y renderizado de interfaces adaptado al rol del usuario.*

3. OBJETIVOS DEL SISTEMA

Objetivo General: Desarrollar e implementar un sistema web desacoplado y ligero bajo arquitectura MVC en JavaScript nativo que centralice la administración de usuarios, la validación de membresías en tiempo real, el control del aforo y el monitoreo físico de los afiliados.

Objetivos Específicos:

1. **OE1 (Modelo de Dominio):** Diseñar e implementar estructuras de datos independientes con persistencia en memoria (`store.js`) para gestionar clientes, suscripciones, asistencias y evolución biométrica.
2. **OE2 (Vistas por Rol):** Implementar componentes de interfaz dinámicos y específicos para Administrador, Entrenador y Cliente mediante manipulación nativa del DOM, sin frameworks externos.
3. **OE3 (Reglas y Cumplimiento):** Automatizar la verificación de vencimiento de planes, el aforo máximo de seguridad, el cálculo de IMC y el registro de autorización de Habeas Data.

4. ACTORES DEL SISTEMA Y CASOS DE USO

Actor	Responsabilidad en el Sistema	Casos de Uso Principales (CU)
Administrador	Control financiero, altas de usuarios, gestión de planes y auditoría global.	• **CU01:** Registrar y actualizar clientes y planes. • **CU02:** Validar pagos y estados de suscripción. • **CU03:** Auditar aforo de sala y bitácora de acceso.
Entrenador	Supervisión de salud física, prescripción deportiva y clases dirigidas.	• **CU04:** Registrar mediciones de peso/talla y calcular IMC. • **CU05:** Asignar y actualizar rutinas de entrenamiento. • **CU06:** Controlar asistencia y cupos en clases grupales.
Cliente / Socio	Consulta de estado de cuenta, rutina diaria y consentimiento legal.	• **CU07:** Consultar vigencia de membresía y pagos. • **CU08:** Visualizar rutina asignada e histórico de IMC. • **CU09:** Registrar aceptación de Ley 1581 de 2012.

5. ESPECIFICACIÓN DE REQUERIMIENTOS

Requerimientos Funcionales (RF)

- **RF01 (Control de Acceso):** Bloquear el ingreso si la fecha actual sobrepasa el vencimiento de la suscripción.
- **RF02 (Aforo Operativo):** Restringir nuevos registros al alcanzar la capacidad máxima permitida.
- **RF03 (Cálculo Antropométrico):** Computar automáticamente el IMC y catalogarlo según la escala oficial.
- **RF04 (Tratamiento de Datos):** Registrar marca temporal y consentimiento explícito bajo la Ley 1581 de 2012.

Requerimientos No Funcionales (RNF)

- **RNF01 (Autonomía Técnica):** Servidor HTTP nativo sin dependencias pesadas de terceros.
- **RNF02 (Separación de Capas):** Desacoplamiento estricto del MVC: la vista desconoce las consultas directas al store.
- **RNF03 (Eficiencia en Memoria):** Latencia menor a 10 ms en lecturas y mutaciones de estado.
- **RNF04 (Interfaz Modular):** Estilos CSS puros organizados por componente sin dependencias externas.

6. ARQUITECTURA DE SOFTWARE Y ESTRUCTURA DE PAQUETES

La solución adopta una segmentación estricta de responsabilidades: los **Modelos** administran el estado reactivo, las **Vistas** ejecutan la inyección de componentes en el DOM y el **Controlador** orquesta la captura centralizada de eventos del usuario.

```
proyecto-01-clase/
├── index.js                    # Servidor HTTP nativo (puerto 3000) y ruteo estático
└── src/
    ├── models/                 # CAPA MODELO (M): Lógica de negocio y persistencia
    │   ├── store.js            # Almacén de datos en memoria (single source of truth)
    │   ├── ClienteModel.js     # Entidad usuario, validación de pago y consentimiento
    │   ├── PlanModel.js        # Definición de tarifas, periodicidad y membresías
    │   ├── AsistenciaModel.js  # Conteo de aforo instantáneo y logs de ingreso/salida
    │   ├── RutinaModel.js      # Catálogo de ejercicios, series, cargas y descansos
    │   ├── ClaseModel.js       # Horarios de sesiones y control de cupos grupales
    │   └── EvolucionModel.js   # Histórico de evaluaciones físicas y cálculo de IMC
    ├── views/                  # CAPA VISTA (V): Renderizado dinámico del DOM
    │   ├── Vista.js            # Interfaz base compartida para inyección al DOM
    │   ├── VistaAdmin.js       # Interfaz de gestión general, tesorería y aforo
    │   ├── VistaEntrenador.js  # Formulario técnico de evolución física y rutinas
    │   ├── VistaCliente.js     # Tablero de autogestión de perfil y entrenamientos
    │   └── styles.css          # Hoja de estilos desacoplada y modular
    └── controllers/            # CAPA CONTROLADOR (C): Coordinación y eventos
        └── Controlador.js      # Delegación global de eventos (submit/click) y puente M-V
```

7. IMPLEMENTACIÓN DE MÓDULOS CRÍTICOS Y REGLAS DE NEGOCIO

Las reglas operativas del gimnasio se ejecutan en métodos atómicos dentro de la capa Modelo antes de modificar el estado global:

- **Validación de Membresías:** Al solicitar acceso, se evalúa **`fechaCorte >= fechaActual`**. Si la mensualidad expiró, el sistema niega la entrada y señaliza el cobro pendiente.
- **Control Dinámico de Aforo:** Se computa **`(totalIngresos - totalSalidas) < aforoMaximo`**, bloqueando el acceso una vez completada la capacidad de la sala.
- **Cálculo de IMC con Protección de División:** Fórmula de Quetelet con asignación segura de parámetros por defecto:

```
// src/models/EvolucionModel.js: Algoritmo de cálculo de IMC
calcularIMC(pesoKg = 70, tallaMetros = 1.70) {
  if (!tallaMetros || tallaMetros <= 0) return 0;
  const imc = Number((pesoKg / (tallaMetros * tallaMetros)).toFixed(2));
  let clasificacion = "Normal";
  if (imc < 18.5) clasificacion = "Bajo peso";
  else if (imc >= 25 && imc < 30) clasificacion = "Sobrepeso";
  else if (imc >= 30) clasificacion = "Obesidad";
  return { imc, clasificacion };
}
```

8. ESTRATEGIA DE VERSIONAMIENTO Y FLUJO EN GIT/GITHUB

El equipo adoptó un flujo de ramas con revisiones de integración lideradas por Harrison Ospina Salazar:

- **Segmentación por Capas Técnicas:** Desarrollo en ramas específicas (**`feat/model-*`** para el dominio, **`feat/view-*`** para la interfaz y **`feat/controller-*`** para el ruteo de eventos) para evitar colisiones en el repositorio.
- **Commits Semánticos:** Registro granular bajo prefijos convencionales (**`feat:`** funcionalidad, **`fix:`** corrección, **`docs:`** documentación, **`refactor:`** mantenimiento).
- **Revisión de Integración:** Verificación cruzada entre pares antes de fusionar los cambios hacia la rama principal (**`main`**).

9. GUÍA RÁPIDA DE DESPLIEGUE Y EJECUCIÓN

```
# 1. Clonar el repositorio oficial
git clone https://github.com/harrisonospina97/proyecto-01-clase.git

# 2. Acceder al directorio e iniciar el servidor HTTP nativo
cd proyecto-01-clase && node index.js

# 3. Acceder en el navegador: http://localhost:3000
```

10. CONCLUSIONES Y EVALUACIÓN DE INGENIERÍA

1. **Robustez del Patrón MVC:** La separación entre vistas y lógica facilitó la evolución de la interfaz de usuario sin modificar los modelos de cálculo ni la persistencia de datos.
2. **Eficiencia Operativa:** Prescindir de paquetes npm externos redujo el peso del proyecto a kilobytes, logrando un inicio de servidor instantáneo y mínimo consumo de memoria en Node.js.
3. **Escalabilidad y Cumplimiento Normativo:** El módulo de tratamiento de datos garantiza la conformidad legal con la Ley 1581 de 2012, y el diseño del almacén centralizado permite migrar en el futuro hacia una base de datos relacional (SQL) sin alterar la capa de presentación.